Programming Assignment 1
Introduction to Socket Programming

1 Introduction

In this project, we would implement a simple TCP layer load balancer (in essence similar to HAProxy (`haproxy.org`). The main objectives of this assignment can be summarized as follows: (1) hands-on experience with "UNIX" sockets, (2) deep understanding of HTTP protocols, and (3) an introduction to system design, and performance monitoring.

2 Multi-threaded Load Balancer

2.1 Specifications

Your balancer server should accept incoming connection requests, and select one of the backend servers (according to a policy) to forward the request to. The response is forwarded back to the client. A log entry would be added including the following information: (a) client ip, (b) client port (c) Request Type (i.e., GET, Post, ..), (d) the chosen server ip, (e) the size of the request, (f) the size of the response, (g) the status code of the response, (h) time of the request and the time of the response.

The load balancer should implement two policies. Policy I is a simple round robin , where servers are chosen in order. Students should propose another policy. You can use the log entries, or propose new metrics to the log.

The solution should be working and running, you an use any backend http webserver (including Apache httpd).

2.2 Notes

- You will want to become familiar with the Socket programming in Java(check out Oracle Socket Tutorial: `https://docs.oracle.com/javase/tutorial/networking/sockets/`.
- You are supposed to handle all return response types.
- the server must support multiple connections concurrently.

3 Bonus

3.1 HTTP 1.1

Using persistent connections to the backend servers, can significantly improve the performance. This can be done by using HTTP 1.1 and reuse the connections.

3.2 X-Forwarded-For

Add X-Forwarded-For to the request to the back end server. For more information, please check this `https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Forwarded-For`.

4 List of Useful Resources

- Oracle Socket Tutorial: `https://docs.oracle.com/javase/tutorial/networking/sockets/`
- KKMultiServerThread.java: `https://docs.oracle.com/javase/tutorial/networking/sockets/examples/KKMultiServerThread.java`
- KKMultiServer.java" `https://docs.oracle.com/javase/tutorial/networking/sockets/examples/KKMultiServe` `java`
- HAProxy http://cbonte.github.io/haproxy-dconv/2.5/intro.html

5 Policy

- You should develop your application in Java (or C/C++, Python) programming language, in teams up to three.
- You are required to submit external documentation for your program and to comment your code thoroughly and clearly. Your documentation should describe the overall organization of your programs, the major functions and data structures.
- No Late submission is accepted.
- You could not use a library (or a class) that provides the load balancer functionality code. (If not sure, ask me)

6 Delivery

- Presentation & Discussion (25%)
  - highlight the key design and solution.
  - list two possible ways to improve the performance. (other than 3.1). You do not need to implement them.
  - You need to show that you understand the solution and the code.
- Report (15%)
  - Mention how you approach the problem.
  - Describe the classes you used, and document them (you may include UML diagrams)
  - User manual: how to run your program
- Code (60%)
  - Your code must compile with no errors
  - HTTP 1.0
- Bonuses
  - Bonus HTTP 1.1 (10%)
  - Bonus add X-Forwarded-For (5%)